EJERCICIO 5 - PARTE 2
=====================

1)
Implementacion de obtenerTodosLosCaminos(origen, destino, grafo).

Es el mismo algoritmo de la parte 1: recorrido en profundidad (DFS) con
backtracking. Esta en DirectedGraphAlgorithms.obtenerTodosLosCaminos.

Recibe:
- un vertice origen (como Comparable, se busca en el grafo)
- un vertice destino (igual)
- un grafo donde cada arista tiene un peso (WeightedEdge)

Devuelve una lista de Path. Cada Path tiene:
- getPath(): los vertices del camino, de origen a destino
- getCost(): el costo total (suma de los pesos de sus aristas)

- Se mantiene un CAMINO ACTUAL (lista de vertices), un COSTO ACUMULADO y un
  conjunto de VISITADOS (los vertices que estan en el camino actual).
- Se parte del origen: se agrega al camino y se lo marca como visitado.
- Si el vertice actual es el DESTINO: se guarda una COPIA del camino con su
  costo y se vuelve atras (no se sigue explorando desde el destino).
  Es una copia porque el camino actual se sigue modificando despues.
- Si no, para cada adyacente del vertice actual que NO este visitado:
    - se lo marca, se lo agrega al camino y se suma el peso de la arista
    - se repite el proceso desde ese vertice
    - al volver (backtracking) se lo saca del camino y se lo desmarca,
      para poder usarlo en otros caminos. El costo no hay que restarlo
      porque se pasa como parametro (cada llamada tiene el suyo).

2)
Precondiciones:
- origen y destino existen en el grafo (si no, se lanza
  IllegalArgumentException).
- los pesos son numeros (pueden ser cualquier valor, no se comparan).

Postcondiciones:
- el grafo no cambia.
- devuelve todos los caminos simples (sin vertices repetidos) de origen a
  destino, en el orden en que los encuentra el DFS.
- si no hay ningun camino, devuelve una lista vacia.
- si origen = destino, devuelve un solo camino [origen] con costo 0.

3)
Pseudocodigo:

```
obtenerTodosLosCaminos(origen, destino, grafo) devuelve Lista de Camino
Comienzo
    SI origen NO ESTA EN grafo O destino NO ESTA EN grafo ENTONCES
        error "el vertice no existe"
    FIN SI
    caminos <- lista vacia
    caminoActual <- [origen]
    visitados <- {origen}
    bpf(origen, 0)
    devolver caminos
Fin
```

```
bpf(actual, costoActual)
Comienzo
    SI actual = destino ENTONCES
        caminos.agregar(copia de caminoActual, costoActual)
        devolver                                  // no se sigue desde el destino
    FIN SI
    PARA CADA arco EN aristasSalientes(actual) HACER
        w <- destino(arco)
        SI w NO ESTA EN visitados ENTONCES
            visitados.agregar(w)
            caminoActual.agregarAlFinal(w)
            bpf(w, costoActual + peso(arco))
            caminoActual.quitarUltimo()           // backtracking
            visitados.quitar(w)
        FIN SI
    FIN PARA
Fin
```

4)
Orden del tiempo de ejecucion:
En el peor caso es exponencial: la cantidad de caminos simples entre dos vertices puede crecer muy rapido (en un grafo completo de n vertices hay del orden de (n-2)! caminos). No se puede evitar, porque hay que listarlos todos. Ademas, guardar cada camino cuesta O(V) por la copia.

5)
EJECUCION: Montevideo -> Porto Alegre (MainEjercicio5)

Se usa el mismo grafo de la parte 1 (Cuadro 2), con los adyacentes en el mismo orden. El programa encuentra los mismos 18 caminos, en el mismo orden que el paso a paso en papel:

#	Camino	Costo
1	M - BA - As - Cu - Ri - SP - PA	6880
2	M - BA - As - Cu - Ri - SP - Sa - PA	7320
3	M - BA - As - Cu - Ri - Sa - PA	7200
4	M - BA - As - Cu - Ri - Sa - SP - PA	7200
5	M - BA - As - PA	2500
6	M - BA - PA	1450
7	M - PA	1000
8	M - SP - As - BA - PA	5030
9	M - SP - As - Cu - Ri - Sa - PA	7580
10	M - SP - As - PA	2880
11	M - SP - Cu - Ri - Sa - PA	6180
12	M - SP - PA	1960
13	M - SP - Ri - Sa - PA	5880
14	M - SP - Sa - PA	2400
15	M - Sa - PA	2400
16	M - Sa - SP - As - BA - PA	5470
17	M - Sa - SP - As - PA	3320
18	M - Sa - SP - PA	2400

Total: 18 caminos.
Menor costo: M - PA que es directo (1000).
Mayor costo: M - SP - As - Cu - Ri - Sa - PA (7580).

6)
CASOS DE PRUEBA (Ejercicio5Test):

- Montevideo -> Porto Alegre en el Cuadro 2: 18 caminos, con los costos y
  el orden de la parte 1.
- Ningun camino repite vertices.
- Grafo con un ciclo (A->B->D, A->C->D, D->A): 2 caminos de A a D, el ciclo
  no genera caminos infinitos.
- Un vertice usado por un camino se puede usar en otro (A->B->D y
  A->C->B->D): comprueba que se desmarca al volver.
- Destino inalcanzable: lista vacia.
- Origen = destino: un solo camino [A] con costo 0.
- Vertice inexistente: IllegalArgumentException.